

Even Hub SDK — Complete Developer Reference

Compiled from <https://hub.evenrealities.com/docs> — April 2026

For sommNI G2 + soPHICON G2 development

1. Overview

The Even Realities G2 are smart glasses with:

- **Display:** 576 × 288 pixels per eye
- **Color depth:** 4-bit greyscale (16 shades of green)
- **Connectivity:** Bluetooth 5.2
- **Audio input:** 4-mic array (single stream, 16kHz PCM signed 16-bit LE mono)
- **Input:** Touchpads on temples (press, double-press, swipe up/down) + optional R1 ring (same gestures)
- **No:** camera, speaker, background colors, font control, text alignment, animations, arbitrary pixel drawing, audio output

App logic runs on the phone (in a WebView); the glasses handle display rendering and native scroll processing.

Architecture:

Even Hub Cloud ↔ (HTTPS) ↔ Phone (WebView) ↔ (Bluetooth) ↔ G2 Glasses

The SDK injects a JavaScript bridge (**EvenAppBridge**) into the WebView.

- **Web** → **Glasses:** `bridge.callEvenApp(method, params)`
 - **Glasses** → **Web:** Input events trigger `window._listenEvenAppMessage(...)`
-

2. Installation

Prerequisites

- Node.js v18+
- Web framework (Vite recommended)
- Even Realities App on phone (for testing)
- G2 glasses (for hardware testing)
- R1 ring (optional)

SDK

```
npm install @evenrealities/even_hub_sdk    # v0.0.9
```

Typed methods for display control, input handling, audio, device info, and local storage.

Simulator

```
npm install -g @evenrealities/evenhub-simulator # v0.6.2
```

Cross-platform (macOS, Linux, Windows). Supplements but does not replace hardware testing.

CLI

```
npm install -D @evenrealities/evenhub-cli # v0.1.10
```

Handles authentication, QR sideloading, and app packaging.

3. Your First App

Initialize the SDK

```
import { waitForEvenAppBridge, EvenAppBridge } from '@evenrealities/even_hub_sdk'

// Recommended: async wait – resolves when the bridge is ready
const bridge = await waitForEvenAppBridge()

// Alternative: synchronous singleton – only after bridge is initialized
const bridge = EvenAppBridge.getInstance()
```

Create a Page

```
import { waitForEvenAppBridge, TextContainerProperty } from '@evenrealities/even_hub_sdk'

const bridge = await waitForEvenAppBridge()

const textContainer = new TextContainerProperty({
  xPosition: 0,
  yPosition: 0,
  width: 576,
  height: 288,
  borderWidth: 0,
  borderColor: 5,
  paddingLength: 4,
  containerID: 1,
  containerName: 'main',
  content: 'Hello from G2!',
  isEventCapture: 1,
})

const result = await bridge.createStartupPageContainer(1, [textContainer])
// result: 0 = success, 1 = invalid, 2 = oversize, 3 = out of memory
```

Run It

```
# With simulator
evenhub-simulator http://localhost:5173

# On real hardware
evenhub qr --url "http://192.168.1.100:5173"
```

4. App Structure

```
my-app/
  ■■■ src/           (main.ts, components/)
  ■■■ public/        (assets/)
  ■■■ index.html
```

```
■■■ package.json
■■■ vite.config.ts
■■■ tsconfig.json
■■■ app.json ← Even Hub manifest (required)
```

Testing Approaches

1. **QR sideloading** — local dev server + CLI QR codes, hot reload
 2. **Private builds** — package via CLI, upload to developer portal
 3. **Simulator** — preview on computer without hardware
 4. **PWA** — host independently, bypass Even Hub packaging/review
-

5. Page Lifecycle

Core Methods

Method	Purpose	Notes
<code>createStartupPageContainer</code>	Create initial page	Called once at startup. Returns result code (0=success, 1=invalid, 2=oversize, 3=OOM)
<code>rebuildPageContainer</code>	Replace entire page	Full redraw, state loss, brief flicker
<code>textContainerUpgrade</code>	Update text in-place	Faster, flicker-free. Container ID + name must match exactly
<code>updateImageRawData</code>	Update image container	No concurrent sends allowed
<code>shutdownPageContainer</code>	Exit the app	Accepts parameter for immediate or confirmed exit
<code>callEvenApp</code>	Generic method call	Wrapper foundation for typed methods

Result Codes

- **Startup:** 0=success, 1=invalid params, 2=oversize, 3=out of memory
- **Rebuild/Upgrade/Shutdown:** boolean
- **Image update:** "success", "imageException", "imageSizeInvalid", "imageToGray4Failed", "sendFailed"

Best Practices

- Use `textContainerUpgrade` for frequent text updates (counters, status, live data)
 - Use `rebuildPageContainer` when changing the container layout
 - Container ID + name must match precisely during text upgrades
 - Image updates must be sequential — no concurrent sends
-

6. Display & UI System

Canvas

- 576 × 288 px per eye
- Origin: top-left (0, 0)
- 4-bit greyscale (16 levels of green)

Container Architecture

UI = rectangular containers with absolute positioning.

Constraints:

- Max **4 image containers** and **8 other containers** per page
- Exactly **one container** must have `isEventCapture: 1`
- Containers overlap based on declaration order (no z-index)

Shared Container Properties

All containers: `xPosition`, `yPosition`, `width`, `height`, `containerID`, `containerName`, `isEventCapture`

Text/List only: `borderWidth`, `borderColor`, `borderRadius`, `paddingLength`

Text Containers (`TextContainerProperty`)

- Plain, left-aligned text only — no formatting options
- Content limits: 1,000 chars at startup, 2,000 for upgrades
- Text wraps at container width
- `\n` for line breaks
- ~400–500 characters fill a full-screen text container
- Update via `textContainerUpgrade` (flicker-free) when layout unchanged

List Containers (`ListContainerProperty`)

- Native scrollable lists
- Max **20 items**, max **64 characters** per item
- **Cannot be updated in-place** — requires full page rebuild
- Configured via `ListItemContainerProperty`:
- `itemCount`: number of items
- `itemWidth`: 0 for auto
- `isItemSelectBorderEn`: 1 to show selection border
- `itemName`: string array of item labels

Image Containers (`ImageContainerProperty`)

- Accepts greyscale images
- Size range: 20–200px width, 20–100px height
- **Cannot send images during `createStartupPageContainer`** — use placeholder, then push via `updateImageRawData`
- Image data: raw grayscale PNG bytes as `number[]`

Unicode Support

The glasses use a single embedded LVGL font. Useful characters:

- Box-drawing elements for borders/separators
 - Progress bar symbols (block characters)
 - Navigation arrows for UI construction
-

7. Input & Events

Input Sources

- **G2 touchpads** (temple): Press, double press, swipe up/down
- **R1 touchpads** (ring): Same gesture set, distinguishable by source
- **IMU sensors**: Head orientation and motion data

Event Types (OsEventTypeList)

Event	Value	Description
CLICK_EVENT	0	Single press
SCROLL_TOP_EVENT	1	Swipe up / scroll reaches top
SCROLL_BOTTOM_EVENT	2	Swipe down / scroll reaches bottom
DOUBLE_CLICK_EVENT	3	Double press
FOREGROUND_ENTER_EVENT	4	App comes to foreground
FOREGROUND_EXIT_EVENT	5	App goes to background
ABNORMAL_EXIT_EVENT	6	Unexpected disconnect

Handling Events

```
bridge.onEvenHubEvent((event: EvenHubEvent) => {  
  if (event.listEvent) {  
    // List scroll/click events  
    const idx = event.listEvent.currentSelectItemIndex;  
    const type = event.listEvent.eventType;  
  }  
  if (event.textEvent) {  
    // Text container events  
    const type = event.textEvent.eventType;  
  }  
  if (event.sysEvent) {  
    // System events (double-click, lifecycle)  
    const type = event.sysEvent.eventType;  
  }  
  if (event.audioEvent) {  
    // Microphone audio data  
    const pcm = event.audioEvent.audioPcm;  
  }  
});
```

Event Routing

- Only **one container per page** can capture events (**isEventCapture: 1**)

- Events route to text or list containers based on that flag
-

8. Device APIs

Audio Control

```
await bridge.audioControl(true); // Start mic capture
await bridge.audioControl(false); // Stop mic capture
```

PCM format: 16kHz, signed 16-bit LE, mono. Arrives via **audioEvent** callbacks.

IMU (Motion Sensors)

```
bridge.imuControl(frequency); // 100-1000 protocol pacing codes
```

Data arrives as **Sys_ItemEvent** objects with x, y, z axis values.

Device Information

```
const device = await bridge.getDeviceInfo();
// device.model, device.sn, device.status.isConnected(), device.status.batteryLevel

bridge.onDeviceStatusChanged((status) => {
  // status.connectType: Connected, Disconnected, Connecting
  // status.batteryLevel, status.wearingStatus, status.chargingState
});
```

User Information

```
const user = await bridge.getUserInfo();
// user.id, user.name, user.avatar, user.country
```

Local Storage

```
await bridge.setLocalStorage("key", "value");
const val = await bridge.getLocalStorage("key");
```

Known Limitations

No direct Bluetooth access, no arbitrary pixel drawing, no audio output, no text alignment, no font control, no background colors, no per-item list styling, no programmatic scroll position, no animations, no camera, images are greyscale only.

9. Design Guidelines

Display Constraints

- 576 × 288 px canvas
- 4-bit greyscale
- No background fills
- Max 4 image containers, 8 other containers
- One event-capturing container per page

Icon Design

- Design at native resolution
- Keep it simple — recognizable silhouettes
- **Test on actual hardware** — green-tinted greyscale differs from monitor

Common UI Patterns

- **Fake buttons:** Use `>` as cursor indicator
- **Selection highlights:** Toggle `borderWidth` on text containers
- **Multi-row layouts:** Stack text containers vertically
- **Progress bars:** Unicode block characters
- **Page flipping:** Pre-paginate text at ~400–500 character boundaries

Figma

Official design file available (linked from docs site).

10. Simulator Reference

Usage

```
evenhub-simulator [OPTIONS] [targetUrl]
```

Options

Flag	Description
<code>-c, --config <path></code>	Path to config file
<code>-g, --glow</code>	Enable glow effect
<code>--no-glow</code>	Disable glow effect
<code>-b, --bounce <type></code>	Animation type (default or spring)
<code>--list-audio-input-devices</code>	List available audio devices
<code>--aid <device></code>	Choose specific audio input device
<code>--no-aid</code>	Use default audio device
<code>--print-config-path</code>	Print default config file path
<code>-V, --version</code>	Print version

Audio Spec

16,000 Hz sample rate, signed 16-bit LE PCM, 100ms per event (3,200 bytes).

Screenshot (v0.5.0+)

Click screenshot button → RGBA PNG with timestamp filename to current directory.

Caveats

- Display rendering may not perfectly match hardware
 - List scrolling behavior differs from real glasses
 - Image processing is faster without hardware size limits
 - Status events unsupported in simulator; supports Up, Down, Click, Double Click
 - **Always validate on actual hardware before deployment**
-

11. Packaging & Deployment

App Manifest (`app.json`)

Generated via `evenhub init`. Required fields:

Field	Requirements
<code>package_id</code>	Reverse-domain, lowercase letters/numbers only per segment, min 2 segments, no hyphens
<code>edition</code>	"202601"
<code>name</code>	Max 20 characters
<code>version</code>	Semver format
<code>entrypoint</code>	Path to entry HTML
<code>permissions</code>	Array of { <code>name</code> , <code>desc</code> , <code>whitelist?</code> }
<code>supported_languages</code>	Language array

Permission Types

`network`, `location`, `microphone` (variants), `album`, `camera`

Build & Pack

```
npm run build
evenhub pack app.json dist -o myapp.ehpk
```

Distribution

Submit `.ehpk` → users download from Even Hub page → launch from glasses menu or Even Realities App.

Common Validation Errors

- Invalid package IDs (hyphens, uppercase, <2 segments)
 - App name >20 chars
 - Incorrect version format
 - Missing required fields
 - Improper permissions formatting
 - Missing entrypoint file
-

12. CLI Reference

Commands

evenhub login — Authenticate developer account. Optional **--email** parameter.

evenhub init — Generate starter **app.json** manifest.

evenhub qr — Create QR codes for sideloading during development.

```
evenhub qr --url "http://192.168.1.100:5173"
```

Supports HTTPS, external display, caching management.

evenhub pack — Bundle built app into **.ehpk** distribution file.

```
evenhub pack app.json dist -o myapp.ehpk
```

Optional package ID availability check.

Shell Completions

Bash, Zsh, Fish supported.

13. Community Resources

G2 Development Notes

Independently maintained reference: **even-g2-notes** on GitHub

- Architecture deep-dives, full Unicode glyph tables, SDK quirks, error codes
- Reference implementations: chess, reddit, weather, tesla, pong, snake

Even Toolkit

Community component library: **even-toolkit** on GitHub

```
npm install even-toolkit
```

- 55+ React components (Button, Card, NavBar, ListItem, Toggle, Dialog, Toast, BottomSheet, Charts, Calendar...)
- 191 pixel-art icons
- Glasses bridge utilities: **useGlasses** hook, **buildActionBar**, **mapGlassEvent**, canvas renderer, PNG utils, pagination helpers
- Design tokens, light/dark themes
- Typography: **.text-vlarge-title** (24px) → **.text-detail** (11px)

Discord

Even Realities developer community for support, bug reports, discussion.